

OLVM

MANUAL COMPLETO DEL CURSO

Oracle Linux Virtualization Manager

De la arquitectura a la operación

OLVM 4.5 · Preparación del examen 1Z0-1170

Edición 2026 · Material de formación

Sobre este manual

Este documento recoge, ordena y desarrolla el contenido de las cinco jornadas del curso. No es una transcripción cronológica de las clases. Está organizado por materias para que pueda utilizarse como referencia después de la formación y como base para preparar el examen.

El manual utiliza continuamente dos puntos de apoyo:

- la instalación real del aula, con un Engine independiente, dos hosts OLVM anidados, almacenamiento NFS y una red lógica para las máquinas de los alumnos;
- la comparación con VMware vSphere, conocido por los participantes.

La idea que ordena todo el curso

OLVM mantiene el estado deseado y toma decisiones; los hosts ejecutan las máquinas. Entender qué componente decide, cuál ejecuta y por qué camino viajan los datos permite explicar casi cualquier operación y localizar casi cualquier fallo.

El producto cambia con las versiones. Los conceptos permanecen bastante estables, pero los requisitos de sistema operativo, los puertos opcionales y las matrices de compatibilidad deben confirmarse siempre en la documentación correspondiente a la release instalada. Las fuentes se mantienen en ``bibliografia.md``.

Cómo está organizado

El manual tiene cinco partes:

1. **Arquitectura y modelo de gestión.** Explica las piezas y la jerarquía.
2. **Red y almacenamiento.** Sigue el camino de una trama y de un bloque.
3. **Máquinas virtuales.** Recorre creación, discos, plantillas, snapshots, migración, permisos y pools.
4. **Scheduling, HA y rendimiento.** Explica colocación, afinidad, fencing, leases, CPU, NUMA y memoria.
5. **Instalación, observabilidad y recuperación.** Describe Standalone Engine, Self-Hosted Engine, incorporación de hosts, monitorización y copias.

Los apéndices contienen la lectura de la instalación del aula, listas de comprobación y un mapa de objetivos del examen. Las preguntas de clase siguen en sus documentos originales; los simulacros completos están en `../exámenes/`.

Convenciones

Los bloques enmarcados contienen la idea que debe quedar después de explicar un tema.

AVISO: Los avisos señalan una confusión frecuente o una operación con riesgo.

Las comparaciones con vSphere son aproximaciones didácticas. Ayudan a trasladar experiencia, pero no convierten dos arquitecturas distintas en equivalentes exactos.

El caso conductor: la plataforma Curso

La plataforma utilizada durante la formación permite ver todas las capas sin confundirla con un diseño de producción:

Elemento	Implementación del aula
Engine	VM independiente ejecutada fuera del entorno OLVM administrado
Data Center	Curso
Cluster	Curso, compatibilidad 4.7
Hosts OLVM	olvm-host1 y olvm-host2, virtualizados sobre worker3 y worker4
Almacenamiento	Data Domains NFS curso-nfs y curso-nfs-2
Servidor NFS	maestro
Red de las VMs	red lógica alumnos
Salida exterior	bridge br-olvm en los workers y NIC física/USB

La virtualización anidada y la dependencia de un único servidor NFS son útiles para aprender, pero no proporcionan la independencia de fallos de una plataforma empresarial. Dos Storage Domains respaldados por los mismos discos tampoco son dos copias independientes.

Índice

Parte I · Arquitectura y modelo de gestión

1. Qué es OLVM y qué problema resuelve
2. KVM, QEMU, libvirt y VDSM
3. Engine, bases de datos e interfaces
4. Data Center, Cluster, Host y SPM

Parte II · Red y almacenamiento

5. Almacenamiento y Storage Domains
6. Networking de una VM

Parte III · Ciclo de vida de las máquinas

7. Anatomía y creación de una VM
8. Discos, formatos y perfiles
9. Guest Agent, VirtIO y dispositivos
10. Snapshots, plantillas, cloud-init, OVA y pools
11. Migración, usuarios, roles, permisos y cuotas

Parte IV · Scheduling, alta disponibilidad y rendimiento

12. Scheduler, afinidad y etiquetas
13. Alta disponibilidad, fencing, lease y watchdog
14. CPU, NUMA, memoria y E/S

Parte V · Instalación, observabilidad y recuperación

15. Diseño e instalación de Engine y hosts
16. Self-Hosted Engine
17. Eventos, tareas, logs y monitorización
18. Backup, restauración y recuperación
19. Método de diagnóstico

Apéndices

- A. Equivalencias con vSphere
- B. Lectura de la instalación del aula
- C. Listas de comprobación
- D. Mapa del examen 1Z0-1170
- E. Documentos complementarios

Parte I · Arquitectura y modelo de gestión

1. Qué es OLVM y qué problema resuelve

1.1. KVM no es una plataforma de gestión

KVM permite que el kernel de Linux utilice las extensiones de virtualización del procesador. Con QEMU, libvirt y herramientas del sistema puede ejecutarse una VM en un servidor. Eso no resuelve por sí solo la operación de decenas o cientos de hosts:

- inventario común;
- elección automática del host;
- redes lógicas consistentes;
- almacenamiento compartido;
- permisos centralizados;
- migración en vivo;
- alta disponibilidad;
- eventos, auditoría y API.

OLVM añade esa capa de gestión. Su función no es sustituir KVM, sino coordinarlo.

KVM hace posible la virtualización en un host. OLVM convierte muchos hosts en una plataforma administrable.

1.2. Relación entre oVirt, RHV y OLVM

Producto	Papel
oVirt	Proyecto abierto del que procede la arquitectura y buena parte del código
Red Hat Virtualization	Distribución empresarial histórica basada en oVirt
Oracle Linux Virtualization Manager	Solución Oracle que administra hosts Oracle Linux KVM
Oracle VM clásico	Producto anterior basado en Xen; no es OLVM

Esta relación explica por qué algunos objetos, comandos internos y nombres históricos conservan palabras como `ovirt`, `rhev` o `VDS`. También permite usar documentación upstream cuando Oracle remite a ella, pero la matriz soportada la determina Oracle.

1.3. Plano de control y plano de datos

La separación más útil de todo el curso es la siguiente:

Plano	Qué transporta	Participantes principales
Control	órdenes, configuración, estados y resultados	Portal/API, Engine, VDSM, libvirt
Datos de storage	lecturas y escrituras de los discos virtuales	QEMU, host, red de storage, NFS/SAN
Datos de red	tramas generadas y recibidas por la VM	vNIC, TAP, bridge, NIC/bond, red física

Engine no conmuta cada paquete ni lee cada bloque del disco. Una vez arrancada la VM, el tráfico principal fluye entre el host, la red y el almacenamiento.

1.4. Qué ocurre si cae Engine

Las VMs que ya se ejecutan continúan en sus hosts porque QEMU/KVM no necesita una orden de Engine para ejecutar cada instrucción. Lo que se pierde o degrada es el plano de control:

- no se toman nuevas decisiones de scheduling;
- no se gestionan migraciones desde el portal;
- no se aplican cambios administrativos;
- no se coordinan normalmente las recuperaciones que dependen de Engine;
- no se actualiza el estado central hasta que vuelva.

La aplicación puede seguir prestando servicio aunque no pueda administrarse desde OLVM. No debe confundirse continuidad de las VMs con disponibilidad completa de la plataforma.

1.5. Qué ocurre si cae VDSM

VDSM es el interlocutor de Engine en cada host. Si deja de responder, Engine no puede confirmar ni aplicar correctamente operaciones en ese nodo. Las VMs pueden seguir ejecutándose, pero el host acabará apareciendo como no respondiente y una recuperación segura requerirá determinar si continúa escribiendo en storage.

2. KVM, QEMU, libvirt y VDSM

2.1. Las cuatro piezas

Componente	Ubicación	Responsabilidad
KVM	kernel del host	ejecución asistida por hardware de CPU y memoria virtual
QEMU	proceso de usuario por VM	modelo de máquina, dispositivos virtuales y proceso de la VM
libvirt	servicio/API local del host	abstracción y control del ciclo de vida de las VMs
VDSM	agente del host	comunicación con Engine y aplicación de operaciones sobre VM, red y storage

Una VM encendida se materializa como un proceso QEMU y varios threads en un host. La definición de la VM, sin embargo, permanece registrada en Engine incluso cuando la VM está apagada.

2.2. Flujo de arranque

1. El usuario pulsa **Ejecutar** en el portal o invoca la API.
2. Engine valida permisos, estado, configuración, redes y almacenamiento.
3. El scheduler construye la lista de hosts válidos.
4. Los filtros eliminan destinos incompatibles.
5. Los pesos ordenan los destinos restantes.
6. Engine envía la orden a VDSM en el host elegido.
7. VDSM prepara storage y red y solicita a libvirt crear el dominio.
8. libvirt inicia QEMU/KVM.
9. QEMU abre los discos y crea los dispositivos de la VM.
10. VDSM devuelve el estado a Engine.

Que un host tenga CPU libre no demuestra que pueda ejecutar una VM. También deben estar disponibles las redes, los Storage Domains, la memoria, el modelo de CPU y un resultado compatible de las políticas de afinidad.

2.3. VM, dominio y proceso

En OLVM la palabra VM puede referirse a tres cosas relacionadas:

- el objeto administrativo guardado en Engine;
- el dominio libvirt que describe la ejecución;
- el proceso QEMU que existe mientras está encendida.

Down significa que la definición existe, pero no hay un proceso QEMU ejecutándola. Eliminar la VM sí borra el objeto y, según la elección realizada, sus discos.

2.4. Virtualización anidada

En el aula, los hosts OLVM son a su vez máquinas virtuales. El hipervisor exterior expone extensiones de virtualización al invitado, y dentro de ese invitado KVM ejecuta las VMs de los alumnos.

Esto introduce dos capas de CPU, red y almacenamiento. Es correcto para formación, pero altera rendimiento, fencing y lectura de dispositivos físicos. Una NIC vista por `olvm-host2` puede ser una NIC virtual del worker exterior, no una tarjeta del servidor físico final.

3. Engine, bases de datos e interfaces

3.1. Engine

Engine mantiene el inventario y el estado deseado. Entre otras funciones:

- registra Data Centers, Clusters, hosts, VMs, plantillas y redes;
- valida operaciones;
- aplica autorización;
- decide colocación mediante el scheduler;
- coordina migraciones y tareas de storage;
- registra eventos;
- expone GUI y API REST.

Engine no es un hipervisor. En un despliegue Standalone tampoco debe configurarse como host KVM administrado.

3.2. Base `engine` y base histórica

Base	Contenido
<code>engine</code>	configuración actual, inventario, relaciones, usuarios internos, eventos y estado operativo
<code>ovirt_engine_history</code>	muestras históricas preparadas por Data Warehouse para tendencias e informes

Data Warehouse extrae periódicamente información operativa y la transforma para consulta histórica. Una métrica instantánea en el portal y una serie temporal de varias semanas no son la misma fuente ni responden a la misma pregunta.

AVISO: PostgreSQL no es una API de administración. Consultar con cuidado puede ayudar al diagnóstico; modificar manualmente las tablas puede romper integridad y dejar un estado que Engine no sabe interpretar.

3.3. Interfaces de administración

Interfaz	Uso principal
Administration Portal	operación completa por administradores
VM Portal	autoservicio limitado para usuarios de VMs y pools
Monitoring Portal / Grafana	visualización histórica y observabilidad
API REST	integración y automatización
SDK	consumo programático de la API
consola VNC/SPICE/noVNC	acceso al display de una VM

Un usuario puede tener permiso para usar una VM desde VM Portal sin disponer de ningún privilegio administrativo sobre el clúster o el host.

3.4. Estado deseado frente a estado real

Cuando se configura una red o una VM desde Engine se expresa un estado deseado. VDSM y los componentes del host intentan aplicarlo. Un cambio manual realizado por fuera puede funcionar temporalmente y, aun así, ser incorrecto para OLVM:

- Engine no lo conoce;
- una reinstalación o nueva sincronización puede reemplazarlo;
- la validación del host puede detectar divergencia;

- otro host puede tener una configuración distinta.

Por eso una infraestructura de red creada previamente con Ansible puede ser perfectamente válida, pero debe distinguirse de su registro y gestión posterior en OLVM.

4. Data Center, Cluster, Host y SPM

4.1. Jerarquía

Objeto	Pregunta que responde	Equivalencia aproximada en vSphere
Data Center	¿qué almacenamiento y redes lógicas forman el entorno?	Datacenter, con diferencias
Cluster	¿qué hosts son compatibles y comparten políticas?	Cluster
Host	¿dónde puede ejecutarse QEMU?	ESXi host
VM	¿qué carga se define y ejecuta?	Virtual Machine
Storage Domain	¿dónde residen discos, plantillas y metadatos?	Datastore, no idéntico

El Data Center es el ámbito de storage. El Cluster es el ámbito principal de compatibilidad de CPU, migración y scheduling. Un host pertenece a un único Cluster y este a un único Data Center.

4.2. Compatibilidad del Cluster

La versión de compatibilidad controla qué capacidades y formatos puede utilizar el clúster. El tipo de CPU establece un nivel que todos los hosts deben soportar. Configurar el modelo más moderno de un único servidor puede impedir incorporar hosts más antiguos o recibir migraciones.

La estrategia habitual es elegir el mínimo común denominador que cubra el parque real y revisar conscientemente el impacto antes de elevarlo.

4.3. Estados del host

Estado	Lectura operativa
Up	disponible para ejecutar VMs
Maintenance	retirado de la planificación para mantenimiento
Installing	Engine está desplegando o actualizando componentes
Non Operational	comunica, pero incumple algún requisito, por ejemplo una red obligatoria
Non Responsive	Engine no consigue comunicarse con VDSM
Install Failed	el alta o la reinstalación no terminó correctamente

`Non Operational` no equivale a que el servidor esté apagado. `Non Responsive` tampoco demuestra que esté apagado: puede existir una partición de red.

4.4. Storage Pool Manager

En cada Data Center con almacenamiento compartido, Engine asigna a un host el rol SPM. Ese host coordina las operaciones que modifican metadatos del almacenamiento:

- crear, borrar y manipular imágenes de disco;
- crear plantillas y snapshots;
- coordinar metadatos de los Storage Domains;
- asignar determinados recursos en almacenamiento de bloques.

SPM no transporta todas las lecturas y escrituras de todas las VMs. Cada host accede directamente al almacenamiento para ejecutar sus cargas.

SPM es el coordinador del storage, no un proxy de datos.

Si el host SPM deja de estar disponible, Engine elige otro host válido. Las VMs que ya hacen E/S pueden continuar, mientras que nuevas operaciones de metadatos pueden esperar a la transición.

4.5. Cómo identificarlo

Desde el portal se puede consultar el Data Center o la columna/estado SPM de los hosts. También puede revisarse la información expuesta por la API. El rol es dinámico: el host que hoy es SPM no tiene por qué seguir siéndolo después de un mantenimiento o fallo.

Parte II · Red y almacenamiento

5. Almacenamiento y Storage Domains

5.1. Backend, dominio, disco y filesystem

Cuatro capas que no deben mezclarse:

Capa	Ejemplo del aula
Backend físico/lógico	discos y filesystem del servidor <code>maestro</code>
Protocolo de acceso	NFS
Storage Domain OLVM	<code>curso-nfs</code>
Disco virtual	imagen QCOW2/RAW asociada a una VM
Filesystem invitado	XFS/ext4 dentro de AlmaLinux

Ampliar el export NFS no amplía automáticamente el filesystem del invitado. Ampliar el tamaño virtual del disco tampoco hace crecer por sí solo una partición o un volumen lógico dentro de la VM.

5.2. Tipos de acceso

Familia	Ejemplos	Cómo aparece el volumen
Fichero	NFS, Gluster, POSIX compatible	archivos/imágenes dentro de un filesystem compartido
Bloque	iSCSI, Fibre Channel	LUNs gestionadas de forma coordinada

OLVM centraliza metadatos y locking, pero la disponibilidad final depende del backend, sus controladoras, caminos, red y diseño de redundancia.

5.3. Tipos de dominio

El Data Domain almacena discos de VMs, plantillas y snapshots. En formatos modernos uno de los Data Domains figura como maestro desde el punto de vista de coordinación del Data Center, pero todos los dominios de datos activos pueden contener cargas.

Los antiguos ISO Domain y Export Domain pertenecen a flujos históricos. En versiones actuales las imágenes ISO pueden cargarse en Data Domains y las importaciones/exportaciones utilizan mecanismos como Image I/O y OVA.

5.4. Ciclo de vida del dominio

1. Se prepara el backend y el acceso desde todos los hosts.
2. Se crea o importa el Storage Domain en Engine.
3. Se adjunta al Data Center.
4. Se activa.
5. Los hosts validan acceso y locking.
6. Puede recibir discos y plantillas.

Para retirar un dominio se migran o eliminan primero las cargas necesarias, se coloca en mantenimiento, se separa y finalmente se elimina su registro. Borrar el registro y destruir físicamente los datos no son siempre la misma acción.

5.5. NFS en el aula

`curso-nfs` y `curso-nfs-2` están activos y son visibles desde los dos hosts OLVM. Esto permite migrar una VM sin copiar su disco: cambia el host que ejecuta QEMU, pero ambos acceden a la misma imagen mediante NFS.

Los dos dominios dependen del mismo servidor y del mismo soporte físico. Sirven para practicar movimiento y selección, no para demostrar recuperación ante el fallo completo del storage.

5.6. SPM, sanlock y exclusión

SPM coordina metadatos y sanlock proporciona mecanismos de locking sobre recursos compartidos. El objetivo es evitar operaciones incompatibles y dobles escrituras. No se debe eliminar manualmente un lock ni modificar los metadatos porque una operación parezca bloqueada sin haber confirmado primero qué tarea continúa trabajando.

5.7. Perfiles de almacenamiento y QoS

Un perfil de disco relaciona el uso de un Storage Domain con una política de QoS. La VM selecciona un perfil; el perfil puede limitar o priorizar E/S. El QoS es un objeto reutilizable, no un atributo mágico del disco físico.

La cadena es:

```
Storage QoS → Disk Profile → disco virtual → VM
```

El permiso para utilizar un perfil también puede formar parte del control de acceso delegado.

6. Networking de una VM

6.1. Las capas

Elemento	Función
NIC física	conecta el host con la red exterior
Bond	agrupa varias NIC para redundancia o capacidad
VLAN	separa dominios de capa 2 sobre un enlace compartido
Bridge Linux	switch virtual de capa 2 dentro del host
Logical Network	intención de red declarada en Engine
vNIC Profile	política de conexión disponible para una vNIC
vNIC	tarjeta virtual presentada a la VM
TAP/vnet	puerto del host que conecta QEMU con el switch virtual

Una red lógica no es por sí sola un switch físico. Declara el nombre, VLAN, MTU, roles y requisitos que Engine espera desplegar o asociar en los hosts.

6.2. Camino de una trama

En una red basada en bridge Linux, sin OVN, una trama de la VM sigue de forma simplificada este camino:

```

eth0 dentro de la VM
    ↓
vNIC VirtIO emulada por QEMU
    ↓
interfaz TAP/vnet del host
    ↓
bridge Linux
    ↓
NIC, bond o subinterfaz VLAN del host
    ↓
switch físico y LAN

```

El bridge aprende direcciones MAC igual que un switch Ethernet. Para comunicar dos VMs del mismo host puede conmutar localmente. Si están en hosts distintos, las tramas salen por las NIC y atraviesan la red física; no hacen falta túneles para una red L2 que ya existe extremo a extremo.

6.3. TAP, vnet y macvtap

Una TAP es una interfaz virtual de capa 2 que entrega tramas Ethernet entre QEMU y Linux. Libvirt suele nombrarla `vnet0`, `vnet1`, etc. Aparece mientras la VM está encendida y se conecta al bridge correspondiente.

`macvtap`, en cambio, conecta una interfaz de la VM de forma directa sobre una NIC inferior mediante `macvlan/macvtap`. Reduce la función del bridge, pero presenta limitaciones de comunicación host-invitado y no debe interpretarse como otro puerto del bridge si `bridge link` no lo muestra como `master` de ese bridge.

6.4. Cómo leer los comandos

`ip -br link` muestra interfaces, estado y MAC, pero no demuestra por sí solo qué puertos pertenecen a un bridge. Para eso se utiliza:

```

ip link show type bridge
bridge link

```

En `worker4`, la salida:

```

enxf8e43b59dd23 ... master br-olvmm
vnet0             ... master br-olvmm

```

demuestra que ambos son puertos de `br-olvmm`. El propio nombre del bridge aparece en `ip -br link`, pero la pertenencia se deduce de la palabra `master` en `bridge link`.

6.5. Qué crea VDSM

Cuando Engine arranca una VM, VDSM/libvirt crea normalmente la interfaz TAP/vnet de esa ejecución y la conecta a la red prevista. La infraestructura exterior `br-olvm` y su enlace físico se crearon previamente mediante Ansible en el laboratorio. Por tanto:

- Ansible preparó parte de la conectividad exterior;
- Engine conoce la red lógica y el perfil vNIC;
- VDSM crea la conexión efímera de la VM al arrancarla.

Si no hubiera ninguna VM encendida, `vnet0` podría no existir, pero el bridge y su NIC física seguirían configurados.

6.6. Bonds y VLANs

En un host ya administrado, OLVM puede crear bonds, VLANs y asignaciones de redes desde **Host** → **Interfaces de red** → **Configurar redes del host**. VDSM/nmstate y NetworkManager aplican la configuración persistente en Oracle Linux.

En producción también hay que configurar el switch físico. Por ejemplo, un bond 802.3ad necesita un LAG/LACP compatible en los puertos del switch. Engine no puede configurar automáticamente un switch Ethernet externo que no esté integrado con algún sistema de automatización.

6.7. Roles de red

Una red lógica puede asumir diferentes usos:

- gestión `ovirtmgmt`;
- tráfico de máquinas virtuales;
- migración;
- display/consola;
- almacenamiento;
- red predeterminada de rutas, según el diseño.

Una red marcada como **obligatoria** debe estar correctamente desplegada en todos los hosts del clúster; su ausencia puede dejar un host `Non Operational`. Separar migración, gestión, storage y datos de VMs mejora aislamiento y capacidad, pero exige diseñar NICs, bonds, VLANs y switches de forma coherente.

6.8. vNIC Profile, filtros y QoS

La VM no se conecta normalmente directamente a la Logical Network: selecciona un vNIC Profile. El perfil puede determinar:

- red de destino;
- QoS de red;
- filtro, como `vdsml-no-mac-spoofing`;
- passthrough;
- port mirroring;
- migrabilidad y failover de determinados perfiles.

La dirección IP no la asigna el perfil. Se configura dentro del invitado mediante DHCP, cloud-init, NetworkManager u otra herramienta del sistema operativo.

6.9. SR-IOV y OVN

SR-IOV presenta funciones virtuales de una NIC física directamente a las VMs. Reduce procesamiento de software y puede mejorar latencia, pero condiciona migración, disponibilidad de VFs y diseño físico.

OVN/Open vSwitch proporciona redes superpuestas y routing virtual distribuido. Puede reducir la necesidad de extender cada red L2 por todos los switches, pero no elimina la red física subyacente, la conectividad IP del underlay ni los routers y firewalls necesarios para salir a otras redes. El laboratorio del curso utiliza principalmente bridges Linux y no depende de OVN para explicar el flujo básico.

Parte III · Ciclo de vida de las máquinas virtuales

7. Anatomía y creación de una VM

7.1. Una VM es configuración más ejecución

La definición de una VM incluye CPU, memoria, firmware, dispositivos, discos, interfaces, políticas y metadatos. Mientras está apagada, toda esa definición permanece en Engine. Al arrancarla, una parte se traduce a un dominio libvirt y a un proceso QEMU en el host elegido.

Los estados principales son:

Estado	Significado
Down	definida, sin proceso QEMU activo
Powering Up	arranque solicitado y en preparación
Up	ejecución activa en un host
Powering Down	apagado ordenado en curso
Paused	ejecución suspendida temporalmente
Migrating From/To	transferencia de estado entre hosts
Image Locked	operación exclusiva sobre una imagen o cadena de discos
Not Responding	Engine no obtiene el estado esperado del invitado o de la ejecución

Un estado intermedio no debe forzarse manualmente sin confirmar qué tarea lo mantiene. `Image Locked`, por ejemplo, puede ser correcto mientras se crea una plantilla o se consolida un snapshot.

7.2. Clúster, sistema operativo y optimización

El clúster define el conjunto inicial de hosts posibles y determina compatibilidad de CPU y funcionalidades. El tipo de sistema operativo ayuda a elegir dispositivos y valores compatibles; no instala el sistema operativo. **Optimizado para** aplica una familia de valores predeterminados:

- Servidor: sin elementos propios de escritorio y con política adecuada para cargas persistentes;
- Escritorio: características orientadas a puesto de usuario;
- Alto rendimiento: cambia varios parámetros y exige revisar recomendaciones de CPU, memoria y dispositivos.

7.3. Firmware y chipset

Q35 representa una plataforma PCIe moderna. UEFI sustituye al arranque BIOS tradicional y es necesario para determinadas combinaciones, como Secure Boot o TPM virtual. Cambiar firmware después de instalar el invitado puede impedir que encuentre el cargador de arranque.

El valor seleccionado debe mantenerse coherente desde la plantilla hasta las VMs derivadas.

7.4. CPU virtual

El total de vCPUs se presenta mediante una topología virtual de sockets, cores y threads:

```
vCPU totales = sockets virtuales × cores por socket × threads por core
```

Dos topologías con el mismo total no siempre son equivalentes para licenciamiento, NUMA o reconocimiento del invitado. Una vCPU no es un core físico reservado salvo que exista una política de pinning y reserva coherente.

7.5. Memoria definida, máxima y garantizada

Valor	Función
Memoria definida	RAM presentada normalmente al arrancar
Memoria máxima	techo que permite crecimiento/hot plug según soporte
Memoria garantizada	cantidad que el scheduler y la política no deben recuperar por debajo del compromiso

Si definida y garantizada son ambas 1 GiB, activar ballooning crea el dispositivo, pero no deja margen práctico para recuperar memoria por debajo de ese GiB.

7.6. Número de serie, zona horaria y tipo de instancia

El tipo de instancia puede proporcionar una configuración reutilizable de CPU y memoria sin incluir discos. La política de número de serie decide qué identificador ve el invitado; importa para inventario y licencias. La diferencia horaria del reloj de hardware permite adaptar invitados que esperan UTC o una hora local determinada.

7.7. Crear desde cero o desde plantilla

Crear desde cero define hardware virtual y después exige instalar el sistema operativo. Crear desde plantilla parte de un disco y una configuración ya preparados. La plantilla acelera el despliegue, pero obliga a sellar identidades y personalizar cada nueva VM.

8. Discos, formatos y perfiles

8.1. Tamaño virtual y consumo real

Un disco de 100 GiB declara el espacio máximo visible para el invitado. No significa necesariamente que ya ocupe 100 GiB en el backend. El consumo depende de la política de asignación, el formato, los bloques escritos y la cadena de snapshots.

8.2. RAW y QCOW2

Formato	Ventajas	Costes y límites
RAW	estructura sencilla y acceso predecible; buena opción para preasignación y rendimiento	no proporciona por sí mismo las funciones avanzadas de QCOW2
QCOW2	copy-on-write, crecimiento fino, backing files y snapshots	más metadatos y posible fragmentación/overhead

No debe resumirse como «RAW rápido, QCOW2 lento» sin considerar el backend y la política. Un disco QCOW2 sobre NFS, un volumen de bloques preasignado y una cadena de clones tienen recorridos muy diferentes.

8.3. Thin frente a preallocated

- **Thin provision:** consume capacidad según se escriben bloques. Mejora aprovechamiento, pero permite sobreasignación y exige monitorizar crecimiento.
- **Preallocated:** reserva capacidad desde el principio. Reduce incertidumbre y ciertas operaciones de extensión, a costa de ocupar espacio inmediatamente.

Thin provisioning no crea capacidad física. Si todas las VMs utilizan a la vez lo que se les prometió y el backend se agota, la plataforma falla aunque la suma de tamaños virtuales pareciera válida al crearlas.

8.4. Interface VirtIO-SCSI

VirtIO presenta dispositivos paravirtualizados al invitado. VirtIO-SCSI permite un controlador SCSI eficiente, hot plug y numerosas unidades. Multiqueue puede distribuir E/S entre varias colas/CPU, pero debe dimensionarse; habilitar más colas que carga útil solo añade threads y complejidad.

8.5. Añadir y retirar un disco

Procedimiento seguro:

1. Comprobar capacidad y Storage Domain de destino.
2. Crear o adjuntar el disco y elegir perfil, formato e interfaz.
3. Si se usa hot plug, confirmar que invitado y controlador lo soportan.
4. Dentro del invitado, identificar el nuevo dispositivo por tamaño y WWN/serial, no solo por un nombre que puede cambiar.
5. Crear partición/LVM/filesystem o incorporar el volumen según el diseño.
6. Para retirarlo, desmontar, eliminar referencias persistentes y confirmar que no está en uso antes de desconectar desde OLVM.

Desconectar el disco de una VM no siempre lo elimina de storage. Borrar el disco sí destruye su contenido y requiere confirmar alcance.

8.6. Mover y copiar discos

Un disco puede moverse entre dominios compatibles. La operación consume red y E/S, mantiene la imagen bloqueada durante fases del proceso y puede tardar mucho más que la actualización visible en la GUI. Se revisan tareas y eventos antes de concluir que está detenido.

8.7. Perfil de disco

El perfil seleccionado determina qué política de QoS se aplica y quién puede usar el almacenamiento. En el aula, el perfil `curso-nfs` apunta al dominio del mismo nombre sin limitación especial; en producción un perfil puede distinguir clases Gold, Silver o Backup sobre un mismo backend.

9. Guest Agent, VirtIO y dispositivos

9.1. QEMU Guest Agent

`qemu-guest-agent` se ejecuta dentro de la VM y comunica información y acciones a través de un canal virtual. Permite, según sistema y configuración:

- conocer mejor hostname, IP y estado del invitado;
- solicitar apagado o reinicio ordenado;
- coordinar freeze/thaw del filesystem durante determinados snapshots;
- informar de aplicaciones y datos del sistema;
- facilitar algunas operaciones de gestión.

En AlmaLinux/Oracle Linux:

```
dnf install qemu-guest-agent
systemctl enable --now qemu-guest-agent
```

La VM puede ejecutar sin Guest Agent, pero Engine verá peor lo que ocurre dentro.

9.2. VDSM no es Guest Agent

Componente	Dónde corre	A quién administra
VDSM	host KVM	libvirt, QEMU, red y storage del host
QEMU Guest Agent	sistema invitado	información y acciones dentro de una VM

Instalar Guest Agent dentro de una VM no convierte esa VM en host ni permite que Engine administre su Linux como si fuera VDSM.

9.3. Drivers VirtIO

VirtIO es una familia de dispositivos paravirtualizados:

- `virtio-net` para red;
- `virtio-blk` o `virtio-scsi` para disco;
- `virtio-balloon` para ballooning;
- `virtio-rng` para entropía;
- canales seriales para agentes.

Linux moderno suele incluir los drivers en el kernel. Windows normalmente exige proporcionarlos durante o después de la instalación. Los drivers afectan al camino de datos; Guest Agent afecta a la gestión. No son intercambiables.

9.4. Balloon driver

El driver `virtio_balloon` suele venir en el kernel de AlmaLinux; no se instala un paquete llamado `ballooning`. Puede comprobarse con:

```
modinfo virtio_balloon
lsmod | grep virtio_balloon
```

El host pide al driver reservar páginas dentro del invitado. QEMU puede devolver la memoria física correspondiente al host. No es swap y no aumenta la memoria máxima de la VM.

9.5. Watchdog virtual

OLVM puede presentar un dispositivo watchdog, habitualmente `i6300esb`. Dentro de AlmaLinux se instala un daemon que lo alimente:

```
dnf install watchdog
systemctl enable --now watchdog
```

El daemon abre `/dev/watchdog` y envía pulsos. Si deja de hacerlo, el dispositivo expira y QEMU/OLVM ejecuta la acción configurada, por ejemplo reset o apagado.

El paquete no detecta mágicamente todos los fallos de una aplicación. Deben configurarse las comprobaciones que justifican continuar alimentando el watchdog. Tampoco sustituye el fencing del host: actúa dentro del ámbito de una VM.

9.6. Hot plug

Discos y vNIC suelen admitir conexión en caliente. CPU y memoria dependen de la configuración máxima, del sistema operativo invitado y de los límites expuestos al arrancar. Hot-unplug es más delicado: el invitado debe liberar antes el recurso.

Una operación aceptada por Engine puede quedar pendiente hasta el siguiente reinicio. El icono de cambios pendientes debe revisarse antes de asumir que el invitado ya utiliza la nueva configuración.

10. Snapshots, plantillas, cloud-init, OVA y pools

10.1. Snapshot

Un snapshot captura la configuración y el estado de los discos en un punto determinado. Con QCOW2 crea una nueva capa activa y conserva la anterior como referencia.

```
base ← snapshot 1 ← snapshot 2 ← capa activa
```

Las escrituras nuevas van a la capa activa. Para leer un bloque que no está en ella se recorre la cadena hasta encontrarlo.

Un snapshot es una relación entre capas del mismo sistema de almacenamiento. No es un backup independiente.

Si falla el Storage Domain completo, normalmente se pierden tanto la VM como sus snapshots.

10.2. Consistencia y memoria

Un snapshot en vivo puede ser:

- crash-consistent, parecido al estado después de un corte de corriente;
- quiesced, si el Guest Agent coordina freeze/thaw;
- con memoria, si además guarda RAM y estado de dispositivos.

Guardar memoria facilita volver a una ejecución concreta, pero aumenta tamaño, tiempo y dependencia de compatibilidad.

10.3. Preview, Commit y Undo

Para probar una restauración se selecciona un snapshot y se realiza **Preview**. La VM debe estar en el estado requerido por la versión y el tipo de operación; en muchos casos debe estar apagada. Después:

- **Commit** convierte la vista previa en estado permanente;
- **Undo** abandona la vista previa y vuelve a la cadena anterior.

Borrar un snapshot no borra necesariamente «los datos posteriores». OLVM fusiona los bloques necesarios para mantener coherente la cadena de snapshots que permanece. Las capas físicas cambian; el estado lógico de los snapshots no eliminados debe conservarse.

10.4. Plantilla

Una plantilla es una definición reutilizable y una imagen base de solo lectura a partir de la cual se crean VMs. Durante su creación la imagen puede aparecer **Locked** mientras se copia o transforma; no debe forzarse el desbloqueo.

Antes de crearla se sella el sistema:

- eliminar claves SSH de host;
- limpiar `machine-id` según el procedimiento de la distribución;
- eliminar hostname e IP específicos;
- limpiar reglas persistentes y credenciales no reutilizables;
- preparar cloud-init o Sysprep;
- apagar ordenadamente.

10.5. Clon dependiente y clon completo

Modalidad	Disco	Consecuencia
Thin/dependiente	capa copy-on-write que referencia la plantilla	rápido y ahorra espacio; mantiene dependencia de la base
Clone/independiente	copia completa del contenido	tarda y ocupa más; elimina dependencia de la plantilla

Una plantilla no es simplemente «una ISO subida». La ISO instala; la plantilla ya contiene un sistema y una configuración preparados.

10.6. Carga de imágenes

Un ISO o disco de nube puede cargarse en un Data Domain mediante la opción de carga de discos/ISOs y el Image I/O Proxy. Después:

- una ISO se conecta como medio de instalación;
- una imagen de disco se importa o adjunta y puede servir de base para una VM;
- una VM preparada puede convertirse en plantilla.

El formato del fichero de entrada y el formato administrado finalmente por OLVM no deben confundirse.

10.7. OVA frente a disco

Un disco contiene bloques de una unidad virtual. Un OVA es un appliance empaquetado que puede incluir:

- uno o varios discos;
- definición de CPU y memoria;
- interfaces y dispositivos;
- metadatos de la máquina.

Importar un disco obliga a construir la VM alrededor. Importar un OVA intenta recrear una máquina completa, aunque deben mapearse redes, storage y compatibilidad del entorno destino.

10.8. Cloud-init

Cloud-init personaliza una VM Linux durante los primeros arranques. En la GUI se rellenan campos —hostname, usuario, clave SSH, contraseña, red— o se proporciona YAML personalizado. Engine genera el datasource que la VM recibe; no se edita un fichero persistente de cloud-init en el host.

Ejemplo:

```
#cloud-config
hostname: app01
fqdn: app01.curso.local
manage_etc_hosts: true
users:
  - name: alumno
    groups: [wheel]
    sudo: ALL=(ALL) NOPASSWD:ALL
    shell: /bin/bash
    ssh_authorized_keys:
      - ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI... alumno
ssh_pwauth: false
packages:
  - qemu-guest-agent
runcmd:
  - [systemctl, enable, --now, qemu-guest-agent]
  - [dnf, -y, update]
```

El protocolo de red que se selecciona en el formulario suele ser DHCP o estático, según la forma en que la interfaz vaya a obtener su dirección. Debe coincidir con la red real y con el nombre de interfaz esperado dentro del invitado.

10.9. VM Pools

Un pool agrupa VMs clonadas de una misma plantilla y las ofrece a usuarios. Engine crea las VMs que componen el tamaño del pool; cuando un usuario solicita una, se le asigna una VM disponible. El usuario no crea en ese momento un objeto VM nuevo.

- **Automatic:** la asignación se realiza al solicitar una VM.
- **Manual:** un administrador controla la asignación según el flujo disponible.
- **Prestarted VMs:** mantiene un número arrancado para reducir espera.
- **Stateless:** los cambios de la sesión pueden descartarse al devolver la VM.

Las etiquetas de afinidad contienen VMs y hosts, no el objeto pool. Para aplicar una regla a todo un pool se etiquetan sus VMs y el grupo de afinidad utiliza esa etiqueta. Si se amplía el pool, las nuevas VMs deben incorporarse a la etiqueta o automatizarse la operación.

11. Migración, usuarios, roles, permisos y cuotas

11.1. Live migration

Una migración en vivo mueve la ejecución, no necesariamente el disco. Con NFS compartido:

```
Se mueve:      RAM + estado de CPU/dispositivos + proceso QEMU
Permanece:    disco en NFS + definición en Engine
Se recrea:    TAP/vnet y ejecución en el host destino
```

El destino debe cumplir:

- CPU compatible;
- RAM y recursos suficientes;
- acceso a los mismos Storage Domains;
- redes lógicas y perfiles necesarios;
- políticas de migración y afinidad;
- disponibilidad de dispositivos requeridos.

Passthrough de dispositivos, CPU host-passthrough, pinning rígido o SR-IOV no migrable pueden bloquearla.

11.2. Migración planificada frente a HA

La live migration se realiza mientras origen y destino cooperan. En el fallo brusco de un host no puede copiarse la RAM desde un origen muerto; HA reinicia la VM en otro host después de asegurar que el anterior no continúa ejecutándola.

11.3. Privilegio, rol y permiso

Concepto	Definición
Privilegio	acción elemental permitida
Rol	conjunto reutilizable de privilegios
Permiso	unión de usuario/grupo, rol y objeto

Permiso = sujeto + rol + objeto de alcance

Asignar un rol sobre un Data Center puede heredarse hacia objetos inferiores; asignarlo sobre una sola VM limita el alcance. El mismo rol cambia completamente de efecto según el objeto al que se vincule.

11.4. Roles administrativos y de usuario

Los roles administrativos permiten configurar la plataforma y aparecen en el Administration Portal. Los roles de usuario permiten consumir recursos, abrir consola, utilizar VMs o crear elementos dentro de los límites delegados.

No se concede `SuperUser` para resolver cualquier necesidad. Se elige el rol mínimo y el alcance más pequeño que permita la tarea.

11.5. Usuarios y grupos

OLVM puede trabajar con el dominio interno y proveedores de identidad externos. Los grupos permiten administrar permisos de equipos completos en lugar de repetirlos usuario por usuario. La identidad autentica; el permiso autoriza.

Un usuario que inicia sesión correctamente puede no ver ninguna VM si no posee un permiso sobre ella, su pool o un objeto superior apropiado.

11.6. Cuotas

Las cuotas se definen en el ámbito del Data Center y limitan consumo de recursos, por ejemplo:

- memoria y CPU de clúster;
- capacidad de uno o varios Storage Domains.

Pueden utilizar modo de auditoría o de aplicación. El modo auditoría informa sin bloquear; enforcement rechaza operaciones que exceden el límite. Una cuota no es QoS: limita cantidad asignable, no necesariamente rendimiento instantáneo.

Parte IV · Scheduling, alta disponibilidad y rendimiento

12. Scheduler, afinidad y etiquetas

12.1. Filtros, pesos y balanceador

El scheduler no elige simplemente «el host con menos CPU»:

1. obtiene candidatos del clúster;
2. los filtros eliminan hosts imposibles;
3. los pesos puntúan preferencias;
4. el destino con mejor resultado es seleccionado;
5. el balanceador puede iniciar migraciones para mantener la política del clúster.

Una regla obligatoria actúa como filtro; una preferencia actúa como peso. Si los filtros dejan un conjunto vacío, la VM no arranca aunque existan servidores con capacidad física.

12.2. Cuatro relaciones de afinidad

Relación	Positiva	Negativa
VM–VM	ejecutar juntas	ejecutar separadas
VM–host	preferir/obligar ciertos hosts	evitar/prohibir ciertos hosts

Ejemplos:

- dos nodos redundantes: VM–VM negativa;
- aplicación y caché con mucho tráfico: VM–VM positiva;
- software licenciado por servidor: VM–host positiva;
- host reservado para infraestructura: VM–host negativa para cargas generales.

12.3. Soft y hard

Enforcing convierte la regla en obligatoria. Una regla soft puede incumplirse si es necesario para prestar servicio. Una regla hard impide arrancar o migrar cuando no puede satisfacerse.

Las reglas hard deben probarse también en escenarios de mantenimiento y fallo. Una anti-afinidad dura de tres VMs sobre dos hosts es matemáticamente imposible.

12.4. Política del clúster

El grupo de afinidad solo tendrá efecto si la política de scheduling incluye las unidades correspondientes a afinidad VM–VM y VM–host. Antes de culpar a una regla se revisa la política aplicada al Cluster.

12.5. Etiquetas de afinidad

Una etiqueta es un conjunto reutilizable de VMs y/o hosts. La regla continúa viviendo en el grupo de afinidad. En versiones modernas, la etiqueta no significa por sí misma «estas VMs deben ejecutarse en estos hosts».

En el formulario de grupo, VMs y etiquetas comparten selector; lo mismo ocurre con hosts y etiquetas. Por eso el listado muestra columnas `VM Labels` y `Host Labels` aunque el alta no muestre una caja independiente llamada «Etiquetas».

Las etiquetas de afinidad no deben confundirse con:

- tags generales de inventario;
- labels de red utilizados para desplegar Logical Networks sobre NICs.

12.6. Operación en el portal

1. **Compute** → **Clusters** → **cluster** → **Affinity Labels** → **New**.
2. Crear la etiqueta y seleccionar VMs y/o hosts.
3. **Affinity Groups** → **New**.
4. Definir nombre, VM rule, host rule y `Enforcing`.
5. En los selectores añadir VMs individuales o entradas `Label: ....`
6. Guardar y revisar conflictos.
7. Probar con una regla soft antes de convertirla en hard.

La guía operativa ampliada está en `../afinidad_olvm.md`.

12.7. Afinidad durante migración y HA

El scheduler vuelve a evaluar reglas al arrancar, migrar o recuperar una VM. HA no anula automáticamente una regla hard. Una política demasiado restrictiva puede impedir la recuperación que pretendía proteger.

13. Alta disponibilidad, fencing, lease y watchdog

13.1. Qué garantiza HA

Marcar una VM como altamente disponible permite que Engine intente reiniciarla en otro host cuando su ejecución o el host fallan. Normalmente existe interrupción y se pierde la RAM no persistida. HA de infraestructura no convierte una aplicación en fault tolerant.

Para una recuperación fiable hacen falta:

- más de un host compatible y con capacidad;
- storage y redes disponibles desde el destino;
- política de scheduling satisfiable;
- detección del fallo;
- exclusión del host antiguo;
- definición de VM altamente disponible;
- capacidad N+1 y pruebas periódicas.

13.2. Fencing

Cuando Engine deja de recibir respuesta no sabe si el host está apagado o aislado solo de la red de gestión. Podría seguir ejecutando VMs y escribiendo en storage. Arrancar las mismas VMs en otro host produciría doble ejecución y posible corrupción.

Fencing utiliza el dispositivo de gestión fuera de banda —BMC, iLO, iDRAC, IPMI, etc.— para apagar o reiniciar el servidor dudoso. Otro host actúa como proxy para ejecutar el fence agent porque Engine no se conecta necesariamente al dispositivo por sí mismo.

Se configura en:

Compute → Hosts → Edit → Power Management

Se introducen dirección, usuario, contraseña, tipo de agente y opciones, se prueba y se guarda. Un test exitoso hoy no garantiza que las credenciales sigan siendo válidas meses después; deben verificarse periódicamente.

13.3. VM lease

Un VM lease es un bloqueo renovable guardado en un Storage Domain compartido. Su propósito es ayudar a impedir que la misma VM se considere ejecutable en dos hosts durante un fallo ambiguo.

No es:

- el disco de la VM;
- una cuota;
- un alquiler comercial;
- el rol SPM;
- sustituto universal del fencing.

Seleccionar `curso-nfs` como destino del lease no mueve allí el disco: reserva en ese dominio la estructura de coordinación correspondiente.

13.4. Prioridad y acción al reanudar

La prioridad ordena VMs en colas de arranque o migración cuando los recursos son limitados. **Resume behavior** determina cómo actuar después de pausa/hibernación o circunstancias equivalentes; no sustituye la política HA.

13.5. Watchdog

El watchdog comprueba que el invitado o una condición supervisada continúa alimentando un temporizador. Si expira, puede resetear o apagar esa VM. Fencing actúa sobre el host físico; VM lease coordina propiedad de la VM; watchdog actúa desde dentro de la VM. Son capas complementarias.

13.6. Capacidad N+1

Tener dos hosts no basta si ambos están llenos. N+1 significa que, después de perder el componente previsto, queda capacidad para ejecutar las cargas que deben recuperarse. Se valida con memoria garantizada, CPU, redes, storage, dispositivos y reglas de afinidad, no solo sumando RAM instalada.

14. CPU, NUMA, memoria y E/S

14.1. Cores, threads y SMT

Un core físico puede exponer dos threads mediante SMT/Hyper-Threading. Los threads comparten parte de los recursos del core y no equivalen a dos cores completos.

OLVM puede permitir contar threads como cores para aumentar densidad del scheduler. Esto mejora capacidad aparente, pero no duplica rendimiento. Para cargas sensibles se diseña con medición y reservas.

14.2. Overcommit de CPU

Asignar más vCPUs que CPUs físicas es normal porque no todas las VMs consumen al máximo simultáneamente. El riesgo aparece cuando la suma de demanda sostenida supera la capacidad: aumenta espera de CPU, latencia y steal time.

CPU shares expresan prioridad relativa durante contención; no crean CPU ni fijan un límite absoluto por sí solas.

14.3. CPU pinning

El pinning restringe vCPUs a pCPUs concretas. Puede mejorar predictibilidad y localidad, pero reduce flexibilidad y migrabilidad. Una topología manual debe considerar:

- cores y siblings SMT;
- NUMA nodes;
- threads de emulación y de E/S;
- CPUs reservadas para el host;
- topología equivalente en destinos de migración.

14.4. NUMA

NUMA divide un servidor en nodos con CPU y memoria local. Acceder a memoria de otro nodo cuesta más latencia y ancho de banda. Una VM grande que atraviesa varios nodos puede rendir peor si sus vCPUs ejecutan lejos de las páginas que utilizan.

vNUMA presenta topología NUMA al invitado. NUMA pinning intenta relacionar sus nodos virtuales con nodos físicos. No se activa como receta general: se usa para cargas suficientemente grandes y medidas.

14.5. Ballooning, MoM y overcommit de memoria

Ballooning permite recuperar páginas colaborando con el invitado. MoM puede coordinar políticas de memoria del host. La memoria garantizada limita la cantidad que debería recuperarse.

Si el invitado necesita realmente las páginas reclamadas puede hacer reclaim o swap. Sobreasignar memoria mejora densidad solo mientras la demanda real y las garantías estén correctamente dimensionadas.

14.6. KSM

Kernel Samepage Merging busca páginas de memoria con contenido idéntico y las fusiona como una copia compartida de solo lectura. Si una VM modifica una página, copy-on-write crea su copia privada.

KSM puede ahorrar memoria en muchas VMs similares, pero consume CPU para explorar y comparar páginas. Además, las consideraciones de seguridad y predictibilidad pueden desaconsejarlo en determinadas cargas.

KSM no es ballooning:

KSM	Ballooning
deduplica páginas iguales entre procesos/VMs	pide al invitado liberar páginas
no necesita decidir qué aplicación cierre	necesita driver y cooperación del guest

KSM	Ballooning
intercambia CPU por ahorro potencial	puede crear presión dentro de la VM

14.7. Huge pages

Las huge pages reducen el número de entradas de traducción de memoria y pueden mejorar cargas grandes. Exigen reservar y alinear memoria y pueden reducir la flexibilidad de overcommit, ballooning y migración. En cargas Oracle se decide conjuntamente con la configuración de memoria de la aplicación.

14.8. I/O threads y multiqueue

Separar E/S en threads puede evitar que el thread principal de QEMU procese todo. Multiqueue permite varias colas de red o storage y aprovecha paralelismo en VMs con varias vCPUs. El número debe corresponderse con la carga y con la capacidad del backend. Más colas no corrigen un NFS saturado.

14.9. Perfil High Performance

El perfil de alto rendimiento agrupa recomendaciones y valores orientados a latencia y predictibilidad. No garantiza rendimiento automáticamente. Después de seleccionarlo se revisan advertencias sobre:

- pinning y CPU dedicada;
- huge pages;
- NUMA;
- dispositivos y timers;
- migración y HA;
- reserva de memoria.

Optimizar es elegir conscientemente compromisos. No consiste en marcar todas las casillas avanzadas.

Parte V · Instalación, observabilidad y recuperación

15. Diseño e instalación de Engine y hosts

15.1. Diseñar antes de instalar

Una instalación empieza con un inventario, no con `dnf install`. Deben quedar documentados:

- versión de OLVM y matrices soportadas;
- número actual y futuro de hosts;
- CPU, RAM y almacenamiento de Engine;
- FQDN, DNS directo e inverso;
- NTP y zona horaria;
- VLANs, MTU, bonds y roles de red;
- storage para VMs, Hosted Engine y backups;
- BMC y fencing;
- identidad, certificados y acceso administrativo;
- observabilidad, retención y recuperación.

El mínimo técnico permite arrancar; el recomendado debe soportar crecimiento, mantenimiento y fallo de componentes.

15.2. Standalone Engine

Engine se instala en un servidor o VM independiente de los hosts que administra. La VM puede ejecutarse en otra plataforma, pero su ciclo de vida no depende del OLVM que ella misma controla.

Ventajas:

- bootstrap sencillo;
- separación clara del plano de control;
- mantenimiento independiente de los hosts administrados.

Costes:

- necesita infraestructura externa para su disponibilidad;
- debe protegerse y respaldarse de forma independiente;
- una VM externa mal documentada puede convertirse en punto único de fallo.

La VM de Engine del aula, ejecutada en `worker2`, es Standalone desde el punto de vista de OLVM.

15.3. Preparar el sistema de Engine

El flujo soportado parte de una instalación mínima de Oracle Linux y repositorios correspondientes a la versión de OLVM. Se comprueba:

```
hostname -f
getent hosts engine.ejemplo.local
getent hosts host01.ejemplo.local
timedatectl
```

El FQDN definitivo participa en certificados, URLs, SSO y configuración distribuida. Cambiar solo `/etc/hostname` después de desplegar Engine no constituye un renombrado soportado.

El paquete de release configura repositorios y módulos; el paquete central instala los componentes que `engine-setup` configurará. No se habilitan repositorios arbitrarios ni se añaden aplicaciones empresariales al servidor de Engine.

15.4. engine-setup

`engine-setup` es un configurador idempotente y guiado. Decide, entre otras cosas:

- si configura Engine en el sistema;
- base de datos local o remota;
- Data Warehouse y Grafana;
- integración de identidad;
- certificados y FQDN;
- proxy de consola e Image I/O;
- proveedor OVN opcional;
- firewall;
- escala de Data Warehouse.

Puede volver a ejecutarse para reconfigurar componentes soportados. No crea por sí solo todos los hosts, redes y Storage Domains de producción.

Flujo resumido:

1. instalar Oracle Linux mínimo;
2. configurar nombre, DNS y tiempo;
3. habilitar repositorios OLVM soportados;
4. actualizar dentro de la matriz;
5. instalar los paquetes de Engine;
6. ejecutar `engine-setup`;
7. validar portal, API, servicios y bases;
8. realizar un backup inicial fuera del servidor;
9. preparar e incorporar hosts.

15.5. Puertos como flujos

No se memoriza una lista sin dirección ni propósito. El mapa de estudio es:

Origen	Destino	Puerto orientativo	Uso
navegador/API	Engine	TCP 80/443	portal, API y servicios web
Engine	host	TCP 22	bootstrap/instalación inicial cuando se utiliza SSH
Engine/hosts	host	TCP 54321	comunicación con VDSM
hosts	hosts	rango de migración configurado	transferencia de memoria/estado
cliente de consola	host/proxy	rango VNC/SPICE configurado	consola de VMs
cliente/proxy	Engine/host	TCP 54322/54323 según flujo	Image I/O
Engine/DWH	PostgreSQL	TCP 5432	base remota, cuando existe
hosts	NFS/SAN	según protocolo	discos y metadatos compartidos

La tabla no sustituye la guía de firewall de la release. Bases remotas, OVN, Gluster, SNMP, consolas y almacenamiento cambian los flujos necesarios. IPv6 no se deshabilita arbitrariamente aunque el servicio principal utilice IPv4.

15.6. Preparar un host KVM

El host necesita:

- CPU de 64 bits con VT-x/AMD-V y NX habilitados;
- Oracle Linux y kernel soportados;
- instalación mínima y repositorios correctos;
- FQDN y resolución consistente;
- tiempo sincronizado;
- acceso a Engine, redes y storage;
- reserva de recursos para el propio host;
- red de gestión disponible;
- BMC si se requiere fencing.

Un host KVM se dedica a virtualización. Instalar aplicaciones ajenas aumenta dependencias, consumo y superficie de fallo.

15.7. Qué ocurre al añadir un host

Desde **Compute** → **Hosts** → **New**, Engine:

1. valida identidad y conectividad;
2. accede al host por el método de autenticación seleccionado;
3. instala/configura VDSM y componentes necesarios;
4. crea o distribuye certificados;
5. configura firewall si se autorizó;
6. obtiene hardware, CPU, NUMA, NICs y versiones;
7. valida compatibilidad con el Cluster;
8. despliega la red de gestión y comprueba redes obligatorias;
9. conecta storage y activa el host.

Un fallo puede dejarlo `Install Failed` o `Non Operational`. Se consultan eventos y logs de host-deploy antes de repetir ciegamente la operación.

15.8. Operación posterior

Una vez administrado, los cambios persistentes de red se realizan desde Engine o su API. Para mantenimiento planificado:

1. comprobar VMs no migrables;
2. colocar el host en `Maintenance`;
3. confirmar evacuación o apagado de cargas;
4. aplicar mantenimiento;
5. activar el host;
6. verificar redes, storage, VDSM y eventos.

16. Self-Hosted Engine

16.1. Definición precisa

Self-Hosted Engine ejecuta Engine como una VM especial dentro de los hosts OLVM que administra. No basta con instalar Engine en cualquier VM. El despliegue debe crear el dominio de Hosted Engine, su configuración compartida y los agentes de alta disponibilidad.

Para HA se necesitan al menos dos hosts Self-Hosted Engine compatibles. Pueden existir además hosts regulares, pero estos no ejecutan la VM Engine.

16.2. Requisitos particulares

- FQDN e IP exclusivos para la VM Engine;
- DNS directo e inverso desde hosts y clientes;
- red de gestión preparada antes del despliegue;
- almacenamiento compartido accesible por los hosts candidatos;
- export o LUN dedicado a Hosted Engine;
- recursos reservados para que Engine pueda arrancar en más de un host;
- versiones compatibles y homogéneas entre hosts de HA;
- fencing para recuperación segura ante fallos ambiguos.

En un entorno NFS se prepara, por ejemplo:

```
nfs01.ejemplo.local:/exports/hosted-engine
```

El dominio de Hosted Engine se dedica a la VM de Engine; no se utiliza como almacén general de cargas.

16.3. Bootstrap: resolver el huevo y la gallina

El primer host crea una VM provisional sin depender todavía de un Engine existente:

1. instalar Oracle Linux mínimo en el primer host;
2. ejecutar el precheck de OLVM y corregir advertencias;
3. instalar `ovirt-hosted-engine-setup` y `ovirt-engine-appliance`;
4. iniciar `hosted-engine --deploy --4` si se utilizará IPv4;
5. seleccionar Data Center, Cluster, NIC de gestión y appliance;
6. proporcionar FQDN, IP, credenciales y recursos de la VM;
7. crear localmente la VM provisional mediante libvirt;
8. cloud-init configura la appliance y ejecuta `engine-setup` dentro de ella;
9. el nuevo Engine registra el primer host;
10. crea/conecta el Storage Domain de Hosted Engine;
11. traslada la imagen de Engine al almacenamiento compartido;
12. instala y activa los agentes de HA;
13. la VM definitiva queda gestionada como `HostedEngine`.

Comandos iniciales:

```
olvms-pre-check.py
dnf install ovirt-hosted-engine-setup ovirt-engine-appliance
hosted-engine --deploy --4
```

El asistente también puede ejecutarse mediante Cockpit en combinaciones soportadas. La línea de comandos es necesaria en determinados escenarios, como despliegues detrás de proxy.

16.4. Hosts adicionales

Los siguientes nodos se incorporan desde:

```
Compute → Hosts → New → Hosted Engine → Deploy
```

Engine instala la configuración compartida y los servicios HA. El host descubre el dominio y se convierte en candidato a ejecutar la VM Engine.

16.5. Agentes HA

Cada host candidato ejecuta:

```
ovirt-ha-agent  
ovirt-ha-broker
```

Los agentes comprueban almacenamiento, red, salud de Engine y estado de la VM; publican metadatos compartidos y calculan la idoneidad del host. Como no dependen del servicio Engine, pueden decidir dónde arrancar su VM cuando Engine está caído.

Solo debe existir una instancia activa. Locks y metadatos compartidos ayudan a evitar doble arranque.

16.6. Fallo y mantenimiento

En mantenimiento planificado, Engine puede migrar la VM a otro host. En un fallo brusco no se migra RAM desde un origen muerto: después de excluir el host anterior, los agentes arrancan la misma VM desde storage compartido en otro nodo.

Antes de actualizar o manipular Engine se activa mantenimiento global para que los agentes no interpreten su parada como fallo:

```
hosted-engine --set-maintenance --mode=global  
hosted-engine --vm-status
```

Al terminar:

```
hosted-engine --set-maintenance --mode=none
```

Comprobaciones útiles:

```
hosted-engine --check-deployed  
hosted-engine --vm-status  
systemctl status ovirt-ha-agent ovirt-ha-broker  
hosted-engine --console
```

En el portal, la VM Engine, sus hosts y el dominio especial aparecen identificados con una corona.

17. Eventos, tareas, logs y monitorización

17.1. Cuatro conceptos diferentes

Concepto	Pregunta
Estado	¿cómo está el objeto ahora?
Evento	¿qué ocurrió y cuándo?
Tarea	¿qué operación larga sigue ejecutándose?
Log	¿qué detalle técnico registró un componente?

Una VM **Down** es un estado. «No pudo arrancar porque no había host» es un evento. Crear una plantilla puede ser una tarea. La excepción que devuelve VDSM está en un log.

17.2. Empezar en el portal

Antes de abrir varios ficheros se fija:

- objeto afectado;
- hora exacta y zona horaria;
- operación solicitada;
- severidad y mensaje del evento;
- tarea asociada;
- host origen/destino;
- alcance: una VM, un host, un clúster o toda la plataforma.

Sin una línea temporal es fácil relacionar un error antiguo con un síntoma nuevo.

17.3. Mapa mínimo de logs

Componente	Ubicación orientativa	Contenido
Engine	<code>/var/log/ovirt-engine/engine.log</code>	decisiones, validaciones, API, tareas y errores centrales
Engine setup	<code>/var/log/ovirt-engine/setup/</code>	instalación y reconfiguración
host deploy	logs de host-deploy en Engine/host	incorporación y reinstalación de hosts
VDSM	<code>/var/log/vdsm/vdsm.log</code>	operaciones del host, VMs, storage y red
supervdsm	log del servicio correspondiente	operaciones privilegiadas
libvirt	journal y logs libvirt	creación y control de dominios
QEMU	log del dominio en el host	dispositivos, arranque y proceso de una VM
Hosted Engine HA	<code>/var/log/ovirt-hosted-engine-ha/</code>	score, estado y recuperación de Engine
PostgreSQL	journal/log configurado	conexiones, bloqueos y errores de base

Se leen alrededor de la hora y del ID/nombre del objeto. Buscar solamente la palabra **error** produce ruido y omite mensajes que explican la causa.

17.4. ovirt-log-collector

El colector prepara un archivo consistente con información de Engine, hosts y bases. Es útil para soporte y para evitar recopilar logs de horas diferentes. Se revisa el contenido antes de compartirlo porque puede contener nombres, IPs y configuración sensible.

17.5. Estado actual e histórico

Engine mantiene el estado operativo actual. Data Warehouse conserva muestras históricas en `ovirt_engine_history`. Esto permite responder preguntas distintas:

- ¿qué host está saturado ahora? Portal/estado actual.
- ¿a qué hora comenzó la degradación? Histórico.
- ¿crece el consumo semana tras semana? DWH/Grafana.

17.6. Qué monitorizar

Capacidad

- CPU y memoria disponibles y comprometidas;
- crecimiento de Storage Domains;
- snapshots y thin provisioning;
- reserva N+1.

Salud

- Engine, VDSM, bases, DWH y servicios de consola;
- hosts `Non Responsive/Non Operational`;
- Storage Domains inactivos;
- agentes Hosted Engine y fencing.

Rendimiento

- latencia y throughput de storage;
- presión de CPU, memoria y swap;
- tiempos de migración y operaciones de imágenes;
- drops y saturación de red.

Disponibilidad

- copia reciente de Engine;
- restauración probada;
- rutas de storage y red redundantes;
- credenciales de BMC verificadas;
- alertas entregadas y atendidas.

17.7. Grafana, PCP y SNMP

La integración nativa de Grafana consume el histórico de DWH. Un Grafana externo puede utilizar otras fuentes y retenciones; no debe suponerse que muestra exactamente los mismos datos.

Performance Co-Pilot proporciona métricas detalladas del sistema operativo y archivos históricos mediante `pmlogger`. SNMP y correo permiten integrar eventos con herramientas empresariales. Una observabilidad útil combina estado, tendencia y alerta accionable; no se limita a un dashboard bonito.

17.8. PostgreSQL como diagnóstico

Consultas de solo lectura pueden mostrar tamaño de bases, sesiones y bloqueos. Se realizan con cuentas y procedimientos adecuados. Activar logging detallado tiene coste y debe limitarse a una ventana de diagnóstico.

Nunca se resuelve una inconsistencia cambiando a mano una fila de Engine sin un procedimiento soportado.

18. Backup, restauración y recuperación

18.1. Qué protege `engine-backup`

La copia de Engine protege configuración, bases y componentes seleccionados del plano de control. No copia automáticamente todos los discos de todas las VMs. Se guarda fuera del servidor Engine y fuera del mismo fallo común.

Un backup no probado es solo un fichero. Debe registrarse:

- fecha y resultado;
- versión de Engine;
- alcance utilizado;
- cifrado y custodia;
- ubicación externa;
- prueba de restauración;
- RPO y RTO alcanzables.

18.2. Copia básica

La sintaxis exacta se confirma con la versión instalada. Conceptualmente se genera un archivo con el alcance requerido:

```
engine-backup --mode=backup --scope=all \  
  --file=/ruta/externa/engine-backup.tar \  
  --log=/ruta/externa/engine-backup.log
```

Guardar el archivo únicamente en el disco local de Engine no protege frente a la pérdida de ese servidor.

18.3. Restauración

Flujo general:

1. preparar un sistema operativo y versión compatibles;
2. instalar los paquetes necesarios, sin configurar todavía un Engine diferente;
3. transferir copia y log;
4. inspeccionar el backup;
5. ejecutar `engine-backup --mode=restore` con el alcance correcto;
6. ejecutar `engine-setup` para completar configuración;
7. validar certificados, DNS, servicios, bases y portal;
8. verificar conexión con hosts y storage;
9. corregir diferencias de red o base remota según el escenario;
10. realizar una nueva copia después de estabilizar.

Restaurar en un host nuevo no autoriza a cambiar arbitrariamente el FQDN. Si debe cambiarse, se sigue el procedimiento de renombrado soportado.

18.4. Backup de VMs

Snapshots ayudan a obtener un punto consistente, pero el backup requiere sacar o copiar datos a otro dominio/sistema y gestionar retención. Para aplicaciones con bases de datos puede necesitarse coordinación a nivel de aplicación.

OLVM dispone de API de backup incremental para que soluciones especializadas transfieran bloques. El objetivo operativo es poder restaurar, no simplemente tener snapshots antiguos.

18.5. Disaster Recovery

HA resuelve fallos dentro de un sitio; DR contempla pérdida del sitio o de dependencias compartidas. Un diseño activo/pasivo suele requerir:

- replicación de storage;
- copia/restauración de Engine o despliegue coordinado;
- redes y DNS preparados en el destino;
- orden de activación documentado;
- fencing o aislamiento del sitio anterior;
- procedimientos de failover y failback probados.

Replicar corrupción o un borrado no constituye una copia histórica. HA, backup y DR resuelven riesgos diferentes.

18.6. Renombrar Engine

El nombre participa en certificados, URLs y configuración. Se utiliza la herramienta/procedimiento `ovirt-engine-rename` correspondiente a la release, después de preparar DNS y una copia válida. Cambiar solo `hostname` o `/etc/hosts` deja referencias incoherentes.

19. Método de diagnóstico

19.1. Siete pasos

1. **Proteger servicio y datos.** No reiniciar o borrar antes de conocer el alcance.
2. **Definir el síntoma.** «No funciona» no es verificable.
3. **Fijar la línea temporal.** Hora, zona y último estado correcto.
4. **Delimitar el alcance.** VM, host, clúster, red, storage o Engine.
5. **Elegir la capa.** Control, cómputo, invitado, red o almacenamiento.
6. **Formular una hipótesis comprobable.** Una causa que una evidencia pueda confirmar o refutar.
7. **Aplicar el cambio mínimo y verificar.** Registrar resultado y posibilidad de reversión.

19.2. Escalera de evidencias

```
Estado visible
  ↓
Evento y tarea
  ↓
Log del componente que decidió
  ↓
Log del componente que ejecutó
  ↓
Estado del sistema operativo o backend
  ↓
captura/medición específica
```

Se avanza de lo general a lo específico. No se empieza capturando todos los paquetes de todos los hosts.

19.3. La VM no arranca

Orden de preguntas:

1. ¿Está realmente `Down` o bloqueada por una tarea?
2. ¿Qué evento devuelve Engine?
3. ¿Existe algún host elegible después de filtros?
4. ¿Hay memoria/CPU y compatibilidad de CPU?
5. ¿Están disponibles redes y Storage Domains?
6. ¿Hay reglas hard de afinidad contradictorias?
7. ¿VDSM recibió y ejecutó la orden?
8. ¿Qué indica el log QEMU de esa VM?

19.4. La VM está Up pero no tiene IP

Engine conoce la vNIC, no necesariamente la configuración IP interna. Se revisa:

- perfil vNIC y Logical Network;
- TAP conectado al bridge correcto;
- NIC/bond/VLAN del host;
- DHCP o configuración cloud-init;
- interfaz y rutas dentro del invitado;
- firewall y red física.

19.5. La migración está bloqueada

Se comprueban destino compatible, redes, storage, RAM, afinidad, dispositivos passthrough, política de migración, CPU pinning y estado de las imágenes. Que una migración manual funcionara una vez no demuestra que HA pueda recuperar la VM durante cualquier fallo.

19.6. NFS presenta problemas

Se separan:

- conectividad IP;
- resolución de nombre;
- export y permisos;
- versión/opciones de NFS;
- montaje y latencia desde cada host;
- estado del Storage Domain;
- locks y tareas activas;
- capacidad e inodos del backend.

Reiniciar VDSM no repara un servidor NFS saturado.

19.7. Host Non Responsive

Se evita asumir que está apagado. Se comprueba red de gestión, VDSM, host físico, storage y BMC. Si se recuperarán VMs en otro host, primero se necesita exclusión segura mediante fencing/lease según el diseño.

Apéndice A · Equivalencias con vSphere

OLVM	vSphere aproximado	Diferencia que importa
Engine	vCenter Server	Engine usa VDSM/libvirt y objetos propios de oVirt
Host KVM	ESXi host	Oracle Linux completo con KVM, QEMU, libvirt y VDSM
Data Center	vSphere Datacenter	en OLVM es también ámbito importante de storage
Cluster	vSphere Cluster	compatibilidad CPU y scheduler se expresan de forma distinta
Storage Domain	Datastore	formato, SPM y metadatos no son VMFS/NFS de vSphere
Logical Network	Port group/red lógica	no existe un equivalente único a vDS/port group
vNIC Profile	Port group con políticas	es un perfil sobre una Logical Network
Live migration	vMotion	flujo equivalente; requisitos y protocolos difieren
SPM	sin equivalente directo	coordinador de metadatos de storage del Data Center
QEMU Guest Agent	VMware Tools, parcialmente	Guest Agent no incluye toda la pila de drivers VirtIO
Affinity Group	DRS VM/Host Rules	filtros, pesos y enforcing se modelan de forma propia
Fencing	Host isolation/power management	OLVM hace explícito el agente de energía y su proxy
Self-Hosted Engine	VCSA dentro del entorno, con HA específica	agentes Hosted Engine resuelven el bootstrap sin vCenter externo

Apéndice B · Lectura de la instalación del aula

B.1. Plano de control

```

Portal/API
  ↓
Engine en worker2
  ↓
VDSM de olvm-host1 u olvm-host2
  ↓
libvirt → QEMU/KVM

```

La VM de Engine está fuera del OLVM administrado. Por ello es Standalone aunque sea una máquina virtual.

B.2. Disco

```

QEMU en el host elegido
  ↓
red interna
  ↓
curso-nfs o curso-nfs-2
  ↓
servidor maestro

```

El disco no reside localmente dentro de `olvm-host1` o `olvm-host2`. Una migración cambia la ejecución y conserva el acceso al mismo NFS.

B.3. Red

En `worker4` existe:

```

br-olvm
├─ enx8e43b59dd23
└─ vnet0

```

`vnet0` es el TAP de una VM exterior conectada al bridge; la NIC USB/física da salida. Dentro de `olvm-host2`, VDSM crea otros TAP para las VMs OLVM. Por tanto el camino atraviesa dos niveles de virtualización.

La interfaz `macvtap3@enp1s0f0` pertenece a otro camino directo y no forma parte de `br-olvm` salvo que una salida muestre expresamente `master br-olvm`.

B.4. Límites del laboratorio

- no hay BMC real para fencing de los hosts anidados;
- ambos Data Domains comparten backend físico;
- la caída de `maestro` afecta a discos y posibles leases;
- la red exterior depende de bridges creados fuera de Engine;
- la capacidad no representa una plataforma de CaixaBank o Redeia;
- no se demuestra HA real provocando fallos destructivos.

Apéndice C · Listas de comprobación

C.1. Antes de arrancar una VM

- ☐ estado [Down](#) y sin tarea bloqueante;
- ☐ clúster y CPU compatibles;
- ☐ host con memoria y capacidad;
- ☐ Storage Domains activos;
- ☐ redes obligatorias y perfil vNIC disponibles;
- ☐ afinidad satisfactoria;
- ☐ discos no bloqueados;
- ☐ permisos del usuario;
- ☐ consola y cloud-init preparados.

C.2. Antes de migrar

- ☐ origen y destino [Up](#);
- ☐ acceso común al storage;
- ☐ redes equivalentes;
- ☐ CPU compatible;
- ☐ memoria suficiente;
- ☐ sin passthrough no migrable;
- ☐ reglas de afinidad compatibles;
- ☐ política de migración habilitada;
- ☐ observación de eventos y tareas durante la prueba.

C.3. Antes de declarar HA

- ☐ N+1 real;
- ☐ storage y red redundantes;
- ☐ fencing configurado y probado;
- ☐ VM marcada altamente disponible;
- ☐ lease cuando corresponda;
- ☐ afinidad probada durante pérdida de un host;
- ☐ prioridad y orden de recuperación definidos;
- ☐ aplicación probada después de reinicio;
- ☐ monitorización y alertas activas.

C.4. Antes de instalar

- ☐ matriz vigente consultada;
- ☐ FQDN definitivos;
- ☐ DNS directo e inverso;
- ☐ NTP;
- ☐ repositorios soportados;
- ☐ Minimal Install;
- ☐ puertos por flujo;
- ☐ VLANs, MTU y bonds documentados;
- ☐ storage preparado desde todos los hosts;
- ☐ backup y restauración diseñados;
- ☐ BMC/fencing disponible;
- ☐ credenciales custodiadas.

Apéndice D · Mapa del examen 1Z0-1170

El examen vigente tiene 50 preguntas tipo test. El aprobado es 68 %, es decir, 34 respuestas correctas. Los simulacros del curso reproducen 50 preguntas y un límite de 90 minutos.

Bloque	Peso de referencia	Capítulos principales
Arquitectura y componentes	17 %	1–4
Instalación de Engine y hosts	16 %	15–16
Storage y networking	19 %	5–6 y 8
Administración de VMs	10 %	7–10
Usuarios y permisos	9 %	11
Optimización, eventos y logs	20 %	12, 14, 17 y 19
Recuperación	9 %	13 y 18

Las preguntas suelen evaluar:

- qué componente realiza una función;
- requisitos previos de una operación;
- estados y orden de procedimiento;
- consecuencias de una opción;
- diferencia entre conceptos cercanos;
- diagnóstico inicial de un escenario.

No hay puntuación negativa por error. Se responde todo y se reserva tiempo final para revisar preguntas marcadas.

Apéndice E · Documentos complementarios

- ``glosario.md`` — términos del curso.
- ``bibliografia.md`` — fuentes oficiales y upstream.
- ``../flujo_vm.md`` — diagramas del ciclo completo.
- ``../parametros_creacion.md`` — explicación campo a campo del formulario de VM.
- ``../afinidad_olvm.md`` — afinidad y operación en el portal.
- ``../practicas/plantillas-desde-imagen-de-nube.md`` — práctica de plantillas y cloud-init.
- ``../exámenes/README.md`` — instrucciones y simulacros.

Cierre

El operador de OLVM no necesita memorizar cada detalle interno de QEMU, pero sí debe poder seguir una operación de extremo a extremo:

```
intención en Engine
    ↓
decisión del scheduler
    ↓
ejecución por VDSM/libvirt/QEMU
    ↓
acceso directo a red y storage
    ↓
estado, evento, métrica y log
```

Cuando esa cadena está clara, SPM deja de parecer un servidor de datos, una red lógica deja de confundirse con un bridge y HA deja de reducirse a una casilla. Ese es el objetivo real del curso y la mejor preparación para el examen.